第 7 天 · 11/8 · 今日知识点回顾

文件 IO + 异常处理 + CSV/JSON 读写（W4 收尾验收）

目标：写一个脚本：读脏 CSV → 清洗 → 统计 → 写出干净 CSV + 一份 JSON 摘要，全程用 try/except 兜住，中途出错不崩。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 写一个脚本：读脏 CSV → 清洗 → 统计 → 写出干净 CSV + 一份 JSON 摘要，全程用 try/except 兜住，中途出错不崩。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：5 集 • 合计约 79 分钟（边看边在本机敲一遍）

P86	文件的读取操作	约 28 分钟
P88	文件的写出操作	约 10 分钟
P91	了解异常	约 5 分钟
P92	异常的捕获	约 21 分钟
P100	JSON数据格式的转换	约 15 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看） https://raw.githubusercontent.com/jackfrued/Python-100-Days/master/Day21-30/21.文件读写和异常处理.md

视频地址就在下面的「今天做哪些题」网页里，点一下直达分P。

知识点 1/7 • with open 是唯一正确写法

- 一句话：with 会在块结束时自动关文件，不用手写 f.close()。
- 代码：

```
import pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
with open(DATA / 'orders_dirty.csv', encoding='utf-8-sig', newline='') as f:
head = f.readline()
print(head.strip()[:60])
```

★ 三个参数缺一不可：encoding='utf-8-sig' 吃掉 BOM；newline='' 让 csv 模块自己处理换行（不写的话 Windows 上会出现空行翻倍）。模式 'r' 读、'w' 覆盖写、'a' 追加。

知识点 2/7 • 'w' 会瞬间清空文件

- 一句话：open(path, 'w') 一打开就把原文件清成 0 字节，写失败也回不来。
- 代码：

```
out = DATA / '_tmp_test.csv'
with open(out, 'w', encoding='utf-8-sig', newline='') as f:
f.write('a,b\n1,2\n')
print(out.exists(), out.stat().st_size)     # True 8
out.unlink()                                 # 用完删掉
```

★ 往已有文件里补内容要用 'a'；要「读改写」就写新文件再替换，别直接 'w' 覆盖 ——这一步出错，原数据就没了。

知识点 3/7 • try/except：只兜你预期的错

- 一句话：except 后面写具体异常类型，兜不住的错误要让它照常报出来。
- 代码：

```
def to_num(s, default=0.0):
try:
return float(str(s).replace('¥', '').replace(',', ''))
except (TypeError, ValueError) as e:
return default
print(to_num('¥1,899.00'), to_num(''), to_num(None))
```

★ 裸 except: 会把 KeyboardInterrupt（Ctrl+C）和语法错误一起吞掉，程序该停不停 ——永远写具体类型，至少写 except Exception。

这一页只讲「try/except：只兜你预期的错」。讲完就翻页，下一页是另一个知识点。

知识点 4/7 • 文件不存在、编码不对，各报什么错

- 一句话：FileNotFoundError 是路径错；UnicodeDecodeError 是编码错；KeyError 是列名错。
- 代码：

```
try:
open(DATA / '不存在.csv', encoding='utf-8')
except FileNotFoundError as e:
print('路径错：', e)
try:
open(DATA / 'orders_dirty.csv', encoding='utf-8').readline()
except UnicodeDecodeError as e:
print('编码错，要加 -sig：', str(e)[:60])
```

知识点 4/7 • 文件不存在、编码不对，各报什么错（续）

- （本机实测：不带 utf-8-sig 读 orders_dirty.csv 会报
- UnicodeDecodeError: 'utf-8' codec can't decode byte 0xef in position 0 —— 0xef 就是 BOM 的 EF BB BF。）

知识点 5/7 • csv.DictWriter：按表头写出

- 一句话：先 writer.writeheader() 写表头，再 writer.writerow(dict) 一行行写。
- 代码：

```
import csv
out = DATA / '_clean_sample.csv'
with open(out, 'w', encoding='utf-8-sig', newline='') as f:
w = csv.DictWriter(f, fieldnames=['order_id', 'city', 'amount'])
w.writeheader()
w.writerows([{'order_id': '100001', 'city': '北京', 'amount': 1899.0}])
print(out.stat().st_size)
out.unlink()
```

知识点 5/7 • csv.DictWriter：按表头写出（续）

★ fieldnames 里的列名必须和 dict 的键完全一致，否则报 ValueError:

- dict contains fields not in fieldnames。多出来的键用 extrasaction='ignore' 忽略。

知识点 6/7 • JSON：把统计结果存成「人能读、程序能吃」的格式

- 一句话：json.dump(obj, f, ensure_ascii=False, indent=2)。
- 代码：

```
import json
summary = {'rows': 1200, 'paid_rows': 417, 'paid_sum': 422436.0,
```

- 'top_city': {'南京': 60532.0}}

知识点 6/7 • JSON：把统计结果存成「人能读、程序能吃」的格式（续）

```
with open(DATA / '_summary.json', 'w', encoding='utf-8') as f:
json.dump(summary, f, ensure_ascii=False, indent=2)
print((DATA / '_summary.json').read_text(encoding='utf-8')[:80])
(DATA / '_summary.json').unlink()
```

★ JSON 不支持 datetime / Decimal / numpy 类型 —— 直接 dump 会报

- TypeError: Object of type Timestamp is not JSON serializable。
- 先转成字符串或 float（后面 pandas 阶段天天遇到）。

知识点 7/7 • W4 验收脚本的完整骨架

- 一句话：load → clean → summarize → save，四段各一个函数，main 里只做编排。
- 代码：

```
def main():
rows = load_orders(DATA / 'orders_dirty.csv')      # 读
```

- clean, report = clean_orders(rows)   # 洗 + 记清洗账

知识点 7/7 • W4 验收脚本的完整骨架（续）

```
summary = summarize(clean)                          # 汇总
save_csv(clean, DATA / 'orders_clean.csv')          # 写干净数据
save_json({**report, **summary}, DATA / 'orders_summary.json')
print(f'清洗后 {len(clean)} 行，摘要已写出')
```

- （这份 orders_clean.csv / orders_summary.json 就是 W4 的交付物，W5 直接拿来用 ——今天写不好，后面 7 天都在脏数据上打转。）

代码速查：今天这几段，要在本机跑通（1/9）

写一个 orders_clean.py，跑出 orders_clean.csv + orders_summary.json：

```
import csv, json, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
def to_num(s, default=None):
```

try:

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/9）

```
        return float(str(s).replace('¥', '').replace(',', ''))
```

except (TypeError, ValueError):

return default

```
def load_orders(path):
```

```
    with open(path, encoding='utf-8-sig', newline='') as f:
```

return list(csv.DictReader(f))

代码速查：今天这几段，要在本机跑通（3/9）

```
def clean_orders(rows):
```

```
    seen, clean = set(), []
```

```
    dropped = {'dup': 0, 'bad_amount': 0, 'bad_qty': 0}
```

```
    for r in rows:
        oid = r['order_id']
        if oid in seen:
            dropped['dup'] += 1
            continue
        seen.add(oid)
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/9）

```
amt = to_num(r['amount'])
if amt is None:
    dropped['bad_amount'] += 1
    continue
qty = to_num(r['quantity'])
if qty is None or qty <= 0:
    dropped['bad_qty'] += 1
    continue
r['amount'] = round(amt, 2)
r['quantity'] = int(qty)
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/9）

```
        r['status'] = (r['status'] or '').strip().lower()
```

```
        r['city'] = (r['city'] or '').strip() or '（未知）'
```

clean.append(r)

return clean, dropped

```
def summarize(rows):
```

```
    paid = [r for r in rows if r['status'] == 'paid']
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（6/9）

```
    return {'rows_clean': len(rows), 'paid_rows': len(paid),
```

```
            'paid_sum': round(sum(r['amount'] for r in paid), 2),
```

```
            'cities': len({r['city'] for r in rows})}
```

```
def save_csv(rows, path):
```

```
    with open(path, 'w', encoding='utf-8-sig', newline='') as f:
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（7/9）

```
        w = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
```

w.writeheader()

w.writerows(rows)

```
def save_json(obj, path):
```

```
    with open(path, 'w', encoding='utf-8') as f:
```

```
        json.dump(obj, f, ensure_ascii=False, indent=2)
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（8/9）

```
if __name__ == '__main__':
```

```
    rows = load_orders(DATA / 'orders_dirty.csv')
```

clean, dropped = clean_orders(rows)

s = summarize(clean)

```
    save_csv(clean, DATA / 'orders_clean.csv')
```

```
    save_json({**dropped, **s}, DATA / 'orders_summary.json')
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（9/9）

```
print(dropped, s)
```

验收（做到才算过关，这是 W4 的总验收）：

① 脚本一次跑通，data 目录下多出 orders_clean.csv 和 orders_summary.json；

② 打印出来的 dropped 和 s 里每个数字，你都能说出它是怎么来的；

③ 把 clean_orders 里删掉一行 r['city'] = ...，再跑一遍，说清为什么 cities 变了；

④ 能讲清 utf-8-sig 和 utf-8 的差别（用 0xef 那个报错讲）。

最容易错的地方

★ 往已有文件里补内容要用 'a'；要「读改写」就写新文件再替换，别直接 'w' 覆盖 ——这一步出错，原数据就没了。

★ 裸 except: 会把 KeyboardInterrupt（Ctrl+C）和语法错误一起吞掉，程序该停不停 ——永远写具体类型，至少写 except Exception。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：往已有文件里补内容要用 'a'；要「读改写」就写新文件再替换，别直接 'w' 覆盖 ——这一步出错，原数据就没了。
- [ ] 我能不看资料写出：JSON 不支持 datetime / Decimal / numpy 类型 —— 直接 dump 会报
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 8 天 • numpy：ndarray、向量化、广播、布尔索引

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-08-第07天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data